



Apprentissage par renforcement et jumeau numérique

Un agent qui apprend par essais et erreurs peut découvrir seul une loi de commande, à condition de s'entraîner dans une simulation fidèle et de passer prudemment au système réel. Cette fiche pose les bases, entraîne un agent PPO avec Stable-Baselines3 et détaille le chemin de la simulation au robot.

NIVEAU

Avancé

DURÉE

10 h (5 séances de 2 h)

PRÉREQUIS

Cours « Initiation à l'IA pour l'ingénierie » et bonne pratique de Python

OUTILS

Python 3, Gymnasium, Stable-Baselines3 (PPO), PyBullet ou MuJoCo

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- définir agent, environnement, état, action, récompense et politique, et formuler une tâche de commande comme un processus de décision markovien ;
- écrire une boucle Gymnasium et distinguer fin d'épisode (*terminated*) et troncature (*truncated*) ;
- entraîner un agent PPO sur CartPole avec Stable-Baselines3 jusqu'à un retour moyen d'au moins 475 sur 20 épisodes d'évaluation, puis concevoir une récompense et programmer une randomisation de domaine ;
- préparer le passage sur un système réel : sources d'écart, sécurités indépendantes de la politique, protocole d'évaluation.

1. Les notions de base

En apprentissage par renforcement (*reinforcement learning*, RL), personne ne donne la bonne réponse : un **agent**, le programme de commande, agit sur son **environnement** (mécanique, capteurs, simulateur), observe les conséquences et reçoit un nombre, la **récompense**. Il doit découvrir par essais et erreurs comment maximiser la somme des récompenses à venir.

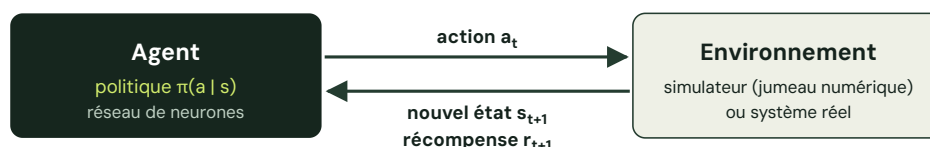


Figure 1 — La boucle agent-environnement, répétée à chaque pas de temps.

À chaque pas de temps t , l'agent reçoit l'état s_t (en pratique une **observation**, partie mesurée de l'état, parfois incomplète ou bruitée), choisit une **action** a_t , discrète ou continue, puis reçoit la récompense r_{t+1} , seule traduction de l'objectif. Sa **politique** $\pi(a | s)$, souvent un réseau de neurones, donne la probabilité de choisir a

dans l'état s . Un **épisode** va de l'état initial à un échec, une réussite ou une limite de durée. Pour un pendule inversé sur un chariot, l'état réunit position et vitesse du chariot, angle et vitesse angulaire du pendule ; l'action pousse à gauche ou à droite ; la récompense vaut +1 par pas où le pendule tient.

1.1 Retour, facteur d'actualisation et valeur

L'agent ne cherche pas la meilleure récompense immédiate, mais le meilleur **retour**, somme actualisée des récompenses à venir :

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Le **facteur d'actualisation** γ , compris entre 0 et 1, règle l'horizon de l'agent, qui vaut environ $1/(1 - \gamma)$ pas : avec $\gamma = 0,99$ et une commande à 50 Hz, l'agent regarde environ 100 pas, soit 2 s, devant lui. La **fonction de valeur** $V^\pi(s)$, retour moyen obtenu depuis l'état s en suivant π , est au cœur de la plupart des algorithmes, dont PPO.

1.2 Le processus de décision markovien

Formellement, la tâche est un **processus de décision markovien** (PDM, en anglais *MDP*) : des états, des actions, une dynamique $P(s' | s, a)$ (probabilité d'arriver en s' après l'action a dans l'état s), une récompense et le facteur γ . On cherche la politique qui maximise le retour moyen. Le simulateur ou le robot jouent le rôle de P : l'agent n'a pas besoin de leurs équations (apprentissage *sans modèle*). L'hypothèse clé est la **propriété de Markov** : l'état présent contient toute l'information utile pour prédire la suite. C'est une affaire de choix des mesures : l'angle seul d'un pendule ne dit pas dans quel sens il tombe, l'angle et la vitesse angulaire, si. Sinon, le problème est *partiellement observable* : on donne à la politique ses dernières observations et actions, ou l'état estimé par un observateur.

2. L'API Gymnasium : `reset` et `step`

Gymnasium (fondation Farama) fixe l'interface entre algorithmes et environnements, simulés ou réels. Installation : `pip install gymnasium stable-baselines3` (PyTorch inclus).

- `env.reset(seed=...)` renvoie `(observation, info)` ; la graine rend l'épisode reproductible. `env.step(action)` applique l'action pendant un pas et renvoie `(observation, récompense, terminated, truncated, info)`. `observation_space` et `action_space` décrivent les valeurs possibles (`Box`, `Discrete(n)`).
- `terminated` signale un état terminal de la tâche (chute, but atteint) ; `truncated`, une interruption extérieure, en général une limite de durée. Après une troncature, la suite aurait continué : Stable-Baselines3 en tient compte dans l'estimation de la valeur.

Dans **CartPole-v1**, un chariot de 1 kg porte un pendule de 0,1 kg et de 1 m (`length`, 0,5 m, est la demi-longueur). L'observation contient x , la vitesse du chariot, l'angle θ et la vitesse angulaire du pendule. Tous les 0,02 s, l'agent applique une force de 10 N vers la gauche (action 0) ou vers la droite (action 1). Il reçoit +1 par pas ; l'épisode se termine si le pendule dépasse 12° ou si le chariot sort de $\pm 2,4$ m, et il est tronqué à 500 pas. On considère la tâche résolue à partir d'un retour moyen de 475.

boucle_gymnasium.py

```
import gymnasium as gym
import numpy as np

env = gym.make("CartPole-v1")
env.action_space.seed(0)
print(env.observation_space.shape, env.action_space) # x, v, θ, ω ; θ = gauche, 1 = droite

def politique_manuelle(obs):
    """Pousse le chariot du côté où le pendule va tomber."""
    x, v, theta, omega = obs
    return int(theta + 0.5 * omega > 0)
```

```
politiques = {"aléatoire": lambda obs: env.action_space.sample(),
             "manuelle": politique_manuelle}
for nom, politique in politiques.items():
    retours = []
    for episode in range(20):
        obs, info = env.reset(seed=episode)
        retour, fini = 0.0, False
        while not fini:
            obs, r, terminated, truncated, info = env.step(politique(obs))
            retour += r
            fini = terminated or truncated # chute ou limite de 500 pas
        retours.append(retour)
    print(f"{nom} : retour moyen {np.mean(retours):.0f} ± {np.std(retours):.0f}")
env.close()
```

Sortie

```
(4,) Discrete(2)
aléatoire : retour moyen 23 ± 13
manuelle : retour moyen 500 ± 0
```

Au hasard, le pendule tombe en moyenne après 23 pas ; la règle manuelle tient les 500 pas des 20 épisodes : CartPole se résout sans apprentissage. C'est un banc d'essai ; le renforcement devient utile quand on ne sait pas écrire la loi de commande (dynamique mal connue, contacts, nombreuses articulations).

3. Entraîner PPO sur CartPole avec Stable-Baselines3

PPO (*Proximal Policy Optimization*, Schulman et al., 2017) est un algorithme **acteur-critique** : un réseau, l'acteur, représente la politique $\pi_\theta(a | s)$ de paramètres θ ; un second, le critique, estime la fonction de valeur. Chaque itération alterne une **collecte**, où la politique actuelle joue `n_steps` pas dans chaque copie de l'environnement, et une **mise à jour** : on calcule pour chaque pas l'avantage $\hat{A}_t$, qui mesure de combien l'action a fait mieux que prévu par le critique (estimation GAE), puis on améliore les deux réseaux en `n_epochs` passes par mini-lots. Pour limiter chaque mise à jour, le rapport $\rho_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ entre nouvelle et ancienne probabilité de l'action jouée est écrêté dans $[1 - \epsilon ; 1 + \epsilon]$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip} \left(\rho_t, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

On maximise L^{CLIP} : une action meilleure que prévu ($\hat{A}_t > 0$) devient plus probable, mais l'objectif ne gagne plus rien au-delà d'un rapport $1 + \epsilon$. La perte totale ajoute l'erreur du critique et, en option, un bonus d'entropie pour l'exploration. Stable, pour actions discrètes ou continues, PPO est un bon premier choix.

entraîner_ppo.py

```
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor

# 4 copies de l'environnement, simulées l'une après l'autre
vec_env = make_vec_env("CartPole-v1", n_envs=4, seed=0)

model = PPO("MlpPolicy", vec_env, seed=0, verbose=1) # réglages par défaut
model.learn(total_timesteps=100_000)
model.save("ppo_cartpole") # crée ppo_cartpole.zip

# Évaluation sur un environnement distinct, politique déterministe
eval_env = Monitor(gym.make("CartPole-v1"))
moyenne, ecart = evaluate_policy(model, eval_env, n_eval_episodes=20,
                                deterministic=True)
print(f"Retour moyen : {moyenne:.1f} ± {ecart:.1f}")
```

Avec Gymnasium 1.4, Stable-Baselines3 2.9 et PyTorch 2.14 sur processeur, `learn` a effectué 13 itérations de $4 \times 2048 = 8192$ pas, soit 106 496 pas, car il termine toujours la collecte en cours. Le retour moyen des épisodes d'entraînement est passé de 22 à 498, et l'évaluation affiche `Retour moyen : 500.0 ± 0.0`. Comptez quelques minutes sans carte graphique ; d'une machine à l'autre, les valeurs intermédiaires varient un peu, même à graine égale. Le hasard pèse lourd : ne comparez deux réglages que sur plusieurs graines.

Réglages par défaut : `n_steps=2048`, `batch_size=64`, `n_epochs=10`, `learning_rate=3e-4`, `gamma=0.99`, `clip_range=0.2` (ϵ). Quatre indicateurs du tableau affiché à chaque itération suffisent :

- `rollout/ep_rew_mean`, retour moyen des derniers épisodes, qui doit monter vers 500 ;
- `train/approx_kl` et `train/clip_fraction`, ampleur des mises à jour : s'ils s'envolent, le pas d'apprentissage est trop grand ;
- `train/explained_variance`, qualité du critique, proche de 1 s'il prédit bien des retours qui varient. Ici, montée à 0,97, elle retombe près de 0 une fois CartPole résolu : les retours à prédire deviennent presque identiques ;
- `train/entropy_loss`, opposé de l'entropie de la politique, qui tend vers 0 quand elle devient déterministe (ici de -0,68 à -0,41).

4. Concevoir la récompense

L'agent optimise la récompense à la lettre. Exemple pour maintenir à la verticale un pendule entraîné par un moteur (angle du pendule θ , nul en haut ; angle du bras α ; commande u normalisée dans $[-1; 1]$) :

Termes d'une récompense par pas et risques associés

Terme	Exemple	Effet recherché	Risque si son poids est mal choisi
Objectif	$\cos \theta$	Redresser et maintenir le pendule	—
Écart secondaire	$-w_1 \alpha^2$	Garder le bras près du centre	Trop fort : le pendule est négligé
Effort	$-w_2 u^2$	Commande modérée, moins d'échauffement	Trop fort : l'agent ne bouge plus
Douceur	$-w_3 (u_t - u_{t-1})^2$	Éviter le broutement de la commande	Trop fort : réactions trop lentes
Échec	-10 en butée, fin d'épisode	Rendre l'échec coûteux	Voir le premier piège ci-dessous

- **Mourir pour ne plus payer.** Si chaque pas coûte et que l'échec termine l'épisode, échouer vite devient la meilleure stratégie : gardez une récompense positive dans la zone souhaitée.
- **Le détournement de récompense.** L'agent optimise ce qui est écrit, pas ce qui était voulu : récompensez seulement une vitesse angulaire faible, et le pendule immobile en bas devient une solution parfaite.
- **Creuse ou dense.** Une récompense creuse (1 en cas de succès, 0 sinon) est fidèle mais rarement rencontrée par hasard ; une récompense dense guide l'exploration mais oriente la solution. Le façonnage par potentiel, qui ajoute $\gamma \Phi(s') - \Phi(s)$, guide l'agent sans changer la politique optimale (Ng, Harada et Russell, 1999).
- **De la méthode.** Grandeurs normalisées, termes ajoutés un par un, et toujours regarder l'agent : la courbe de retour ne dit pas comment il gagne ses points.

5. Jumeau numérique et choix d'un simulateur

Un **jumeau numérique** est le modèle numérique d'un système physique précis, alimenté par ses mesures et recalé sur elles : il ne décrit pas « un pendule » mais ce pendule, avec ses masses, ses frottements, son moteur et ses capteurs. Il offre des millions de pas d'entraînement sans usure ni danger, plus vite que le temps réel. La démarche forme une boucle : mesures, identification des paramètres, jumeau randomisé, entraînement, transfert

prudent, puis nouvelles mesures pour recalculer le modèle. Des équations du mouvement intégrées en Python, comme celles de CartPole, suffisent pour un système simple ; avec plusieurs corps et des contacts, on prend un moteur physique, choisi selon sa fidélité, sa vitesse, les formats de modèle acceptés (URDF exporté de la CAO) et son intégration avec Gymnasium ou ROS 2. Il doit reproduire ce que voit le contrôleur réel : même période de commande, saturation et retard de l'actionneur, seules grandeurs réellement mesurées.

Deux moteurs physiques libres, utilisables depuis Python

Critère	PyBullet	MuJoCo
Installation	<code>pip install pybullet</code> , licence zlib ; modèles URDF, SDF, MJCF	<code>pip install mujoco</code> , licence Apache 2.0 (libre depuis 2022) ; modèles MJCF (natif), URDF
Points forts	Prise en main rapide, nombreux exemples	Contacts rapides et précis, environnements <code>gymnasium[mujoco]</code>
Vigilance	Développement peu actif (dernière version en janvier 2025) ; précompilé pour Linux x86-64 et Python ≤ 3.11 seulement, sinon <code>pip</code> compile les sources (compilateur C++ requis)	Format MJCF à apprendre ; un URDF importé demande souvent des retouches

6. Randomisation de domaine

Un jumeau numérique n'est jamais exact. Plutôt que de chercher le modèle parfait, la **randomisation de domaine** entraîne la politique sur une famille de simulateurs dont les paramètres sont tirés au hasard à chaque épisode : si la famille est assez large, le système réel n'est qu'un tirage de plus. On randomise la dynamique (masses, frottements, couple, retard), les capteurs (bruit, biais) et, pour une caméra, textures et éclairage. Les plages entourent les valeurs identifiées : trop étroites, elles n'apportent rien ; trop larges, elles rendent la politique timorée, voire impossible à apprendre. Ce wrapper tire la masse et la longueur du pendule et la force du moteur à $\pm 30\%$ de leurs valeurs nominales, et ajoute un bruit de mesure.

randomisation.py

```
import gymnasium as gym
import numpy as np

class RandomisationCartPole(gym.Wrapper):
    """Nouveaux paramètres physiques à chaque épisode, plus un bruit de mesure."""

    def __init__(self, env, ecart=0.3, bruit=0.01):
        super().__init__(env)
        self.ecart, self.bruit = ecart, bruit
        p = env.unwrapped
        self.nominal = {"masspole": p.masspole, "length": p.length,
                        "force_mag": p.force_mag}

    def reset(self, *, seed=None, options=None):
        obs, info = self.env.reset(seed=seed, options=options)
        p = self.env.unwrapped
        for nom, valeur in self.nominal.items(): # tirage uniforme à  $\pm 30\%$ 
            facteur = self.np_random.uniform(1 - self.ecart, 1 + self.ecart)
            setattr(p, nom, valeur * facteur)
        p.total_mass = p.masscart + p.masspole # grandeurs dérivées
        p.polemass_length = p.masspole * p.length
        return self._measure(obs), info

    def step(self, action):
        obs, r, terminated, truncated, info = self.env.step(action)
        return self._measure(obs), r, terminated, truncated, info

    def _measure(self, obs): # bruit gaussien sur les quatre mesures
        bruit = self.np_random.normal(0.0, self.bruit, size=obs.shape)
        return (obs + bruit).astype(np.float32)
```

On l'active avec `make_vec_env("CartPole-v1", n_envs=4, seed=0, wrapper_class=RandomisationCartPole)`. Les paramètres restent constants pendant un épisode, le bruit change à chaque pas. CartPole ne calcule `total_mass` et `polemass_length` qu'à sa création : on les met à jour à la main.

7. L'écart simulation-réalité

Une politique excellente en simulation peut échouer sur le système réel : c'est l'**écart simulation-réalité** (en anglais *reality gap*), que la démarche *sim-to-real* cherche à réduire. Le réseau exploite tout le simulateur, y compris ses défauts : une politique « trop bonne » a souvent trouvé une faille du modèle. Sources fréquentes, avec leur symptôme et leurs remèdes :

- **Frottements, jeux, élasticités** (oscillations, erreur statique) : identification sur essais réels, randomisation ;
- **Actionneur**, saturation et zone morte (commande inefficace) : modèle d'actionneur, commande bornée ;
- **Capteurs**, bruit, biais et quantification (commande qui broute) : bruit simulé, filtrage, pénalité de douceur ;
- **Retard et gigue de la boucle** (oscillations croissantes) : même période qu'en réel, retard simulé ;
- **Information privilégiée** (politique inutilisable) : n'observer que ce que les capteurs réels mesurent.

Autres leviers : un historique d'observations et d'actions en entrée de la politique ; une *correction résiduelle* ajoutée à un régulateur classique (PID, retour d'état) plutôt que toute la commande ; quelques essais réels pour finir, toutes sécurités actives.

8. Passer sur le matériel réel en sécurité

Une politique apprise n'offre aucune garantie formelle : face à un état jamais vu, sa sortie peut être n'importe quoi. Les sécurités doivent donc être **indépendantes du réseau de neurones** et organisées en couches.

ATTENTION · MOTEURS, ALIMENTATION ET BATTERIES

- Un **arrêt d'urgence matériel** à portée de main, qui coupe la puissance des moteurs, testé avant chaque séance.
- Un **chien de garde** (*watchdog*) de communication dans le microcontrôleur : sans commande valide pendant quelques périodes, il arrête le moteur, même si l'ordinateur se fige.
- Des **butées** et limites vérifiées par le microcontrôleur, personne dans le volume balayé, puissance augmentée progressivement.
- Une alimentation **limitée en courant** pour les premiers essais, des fusibles ; batteries LiPo chargées sous surveillance en sac ignifugé, jamais déchargées sous leur tension minimale.

Côté logiciel, une couche de sécurité testée à part filtre chaque commande avant l'actionneur : saturation, limitation de la variation par pas, et arrêt verrouillé jusqu'à un réarmement volontaire si une mesure sort du domaine prévu, si la commande n'est pas un nombre fini (NaN ou infini) ou si la boucle prend du retard. La commande nulle n'est pas toujours l'état sûr : un bras soumis à la gravité exige un frein. Cette couche complète l'arrêt d'urgence et le chien de garde, sans les remplacer.

9. Évaluer une politique

Le retour affiché pendant l'entraînement mélange des politiques successives et l'exploration. Une évaluation sérieuse suit un protocole écrit à l'avance : un environnement distinct et la politique **déterministe** ; au moins 20 épisodes et **plusieurs graines d'entraînement** (3 au minimum, 5 à 10 de préférence), avec moyenne et écart-type ou médiane et quartiles ; des **indicateurs métier** (taux de réussite, temps de stabilisation, effort Σu^2 , violations de limites) ; des tests de **robustesse** qui balaient un paramètre physique au-delà des plages d'entraînement ; enfin une **référence** classique (PID, retour d'état LQR) évaluée avec le même protocole, en simulation puis sur le système réel.

EXERCICE GUIDÉ · UNE POLITIQUE ROBUSTE À UN PENDULE MAL CONNU

La randomisation rend-elle la politique plus robuste ? Mesurez-le. Lors de nos essais, avec la graine 0, les politiques nominale et randomisée tenaient 500 pas de 0,5 à 2 fois la longueur nominale du pendule ; à 2,5 fois, la nominale tenait encore presque toujours, la randomisée une centaine de pas. Avec la graine 1, c'est l'inverse : la nominale échouait dès 2 fois, la randomisée tenait encore 500 pas à 2,5 fois. Une graine ne suffit pas pour conclure.

1. Entraînez une politique A avec `entraîner_ppo.py` et une politique B avec en plus `wrapper_class=RandomisationCartPole`, pour les graines 0, 1 et 2, avec le même nombre de pas.
2. Écrivez `env_test(nom, facteur)` : créez un `CartPole-v1`, multipliez l'attribut `nom` de `env.unwrapped` par `facteur`, puis enveloppez-le dans `RandomisationCartPole(env, ecart=0.0)`, qui garde cette valeur, recalcule les grandeurs dérivées et n'ajoute que le bruit de mesure.
3. Évaluez A et B (20 épisodes, `deterministic=True`) pour des facteurs de 0,5 à 3 sur `length` et sur `force_mag`, puis tracez le retour moyen en fonction du facteur : une courbe par politique, moyenne et écart-type sur les trois graines, plage d'entraînement de B (0,7 à 1,3) repérée.

Critères de réussite : chaque politique atteint au moins 475 en conditions nominales pour chaque graine ; la conclusion donne, chiffres à l'appui, la plage de chaque paramètre où chaque politique reste au-dessus de 475 et dit si la randomisation l'a élargie ; le script redonne les mêmes résultats avec les mêmes graines, y compris celle de l'environnement d'évaluation (`reset(seed=...)` avant `evaluate_policy`). Pour aller plus loin : ajoutez un retard d'un pas sur l'action.

À RETENIR

- L'agent maximise le retour actualisé et prend la récompense, seule description de l'objectif, au pied de la lettre.
- PPO est un premier choix robuste, à évaluer sur plusieurs graines ; un jumeau identifié sur le système réel, une randomisation bien dosée et un simulateur qui reproduit retards et saturations réduisent l'écart simulation-réalité.
- Sur le matériel, les sécurités ne dépendent jamais du réseau : arrêt d'urgence, chien de garde, limites dans le microcontrôleur.

Pour aller plus loin

- Stable-Baselines3 : PPO — <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>
- R. S. Sutton et A. G. Barto, *Reinforcement Learning: An Introduction*, 2^e édition, en accès libre — <http://incompleteideas.net/book/the-book-2nd.html>
- J. Schulman et al., « Proximal Policy Optimization Algorithms », 2017 — <https://arxiv.org/abs/1707.06347>
- X. B. Peng et al., « Sim-to-Real Transfer of Robotic Control with Dynamics Randomization », 2018 — <https://arxiv.org/abs/1710.06537>
- Page du cours sur willsim.fr — <https://willsim.fr/cours/apprentissage-renforcement-sim2real.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécanique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr